

C++类和对象完整自学手册

第一部分：基础概念理解

1.1 什么是面向对象编程(OOP)?

面向对象编程是一种特殊的程序设计方法，它将数据和操作数据的方法组合在一起。

OOP的五大特性：

- **抽象**：将复杂的现实世界简化为程序能理解的模型
- **封装**：将数据和方法包装在一起，隐藏内部细节
- **数据隐藏**：防止外部直接访问对象内部数据
- **多态**：同一个操作作用于不同对象时产生不同行为
- **继承**：新类可以继承已有类的特性

1.2 过程性编程 vs 面向对象编程

过程性编程思维：

问题：记录棒球队统计数据
思考过程：
1. 我需要什么步骤？(输入→计算→显示→更新)
2. 如何表示数据？(数组存储各种信息)
3. 用函数来实现各个步骤

面向对象编程思维：

问题：记录棒球队统计数据
思考过程：
1. 我要跟踪什么对象？(选手对象)
2. 对象有什么属性？(姓名、击球数、命中数)
3. 对象有什么行为？(初始化、更新、显示)
4. 用户如何与对象交互？

第二部分：类的基本语法

2.1 类的声明

cpp

```
class Stock // class关键字声明类
{
private: // 私有部分，外部无法直接访问
    std::string company; // 数据成员
    long shares;
    double share_val;
    double total_val;

public: // 公有部分，外部可以访问
    // 构造函数
    Stock();
    Stock(const std::string& co, long n = 0, double pr = 0.0);

    // 析构函数
    ~Stock();

    // 成员函数
    void buy(long num, double price);
    void sell(long num, double price);
    void update(double price);
    void show() const;
}; // 注意分号
```

2.2 访问控制关键字

关键字	含义	使用场景
private	私有，只能在类内部访问	数据成员，内部辅助函数
public	公有，任何地方都能访问	对外接口函数
protected	受保护，类内部和派生类可访问	继承相关（后续章节）

重要原则：

- 数据成员通常放在 中 private
- 对外接口函数放在 中 public
- 默认情况下，类成员是 的 private

2.3 成员函数的实现

```
cpp
```

```
// 在类外部定义成员函数，需要使用作用域解析运算符 ::
void Stock::buy(long num, double price)
{
    if (num < 0) {
        std::cout << "购买数量不能为负数\n";
    } else {
        shares += num;    // 直接访问私有成员
        share_val = price;
        set_tot();        // 调用其他成员函数
    }
}

// 内联函数（在类内部定义）
class Stock {
private:
    void set_tot() { total_val = shares * share_val; } // 自动内联
public:
    // ... 其他成员
};
```

第三部分：构造函数和析构函数

3.1 构造函数详解

构造函数的特点：

- 函数名与类名相同
- 没有返回类型（连void都没有）
- 在创建对象时自动调用
- 可以重载（多个构造函数）

cpp

```

class Stock {
public:
    // 默认构造函数
    Stock() {
        company = "no name";
        shares = 0;
        share_val = 0.0;
        total_val = 0.0;
    }

    // 带参数的构造函数
    Stock(const std::string& co, long n = 0, double pr = 0.0) {
        company = co;
        shares = (n < 0) ? 0 : n;
        share_val = pr;
        total_val = shares * share_val;
    }
};

```

3.2 使用构造函数创建对象

```

cpp

// 方式1：隐式调用
Stock stock1("Apple", 100, 150.0);

// 方式2：显式调用
Stock stock2 = Stock("Google", 50, 2000.0);

// 方式3：默认构造函数
Stock stock3; // 注意：不要写成 Stock stock3();

// 方式4：动态分配
Stock* pstock = new Stock("Microsoft", 200, 300.0);

// 方式5：C++11列表初始化
Stock stock4{"Tesla", 75, 800.0};
Stock stock5{}; // 调用默认构造函数

```

3.3 析构函数详解

析构函数的特点：

- 函数名是类名前加 `~`
- 没有返回类型和参数

- 对象被销毁时自动调用
- 每个类只能有一个析构函数

```
cpp

class Stock {
public:
    ~Stock() {
        std::cout << "Bye, " << company << "!\n";
    }
};

// 析构函数调用时机:
void demo() {
    Stock local_stock("Local", 10, 50.0); // 栈上对象
    // 函数结束时自动调用析构函数
}

Stock* global_stock = new Stock("Dynamic", 20, 100.0);
delete global_stock; // 手动删除时调用析构函数
```

3.4 默认构造函数的重要性

```
cpp

// 如果你定义了带参数的构造函数，就必须也定义默认构造函数
class BadExample {
public:
    BadExample(int x) { value = x; } // 只有带参数的构造函数
private:
    int value;
};

// 这样会编译错误:
BadExample obj;    // 错误! 没有默认构造函数
BadExample arr[10]; // 错误! 数组需要默认构造函数

// 正确的做法:
class GoodExample {
public:
    GoodExample() : value(0) {} // 默认构造函数
    GoodExample(int x) : value(x) {} // 带参数的构造函数
private:
    int value;
};
```

第四部分：this指针

4.1 this指针的概念

`this` 是一个特殊的指针，它指向调用成员函数的对象。

```
cpp

class Stock {
private:
    double total_val;

public:
    void show() const {
        // 以下两种写法等价：
        std::cout << total_val << std::endl;    // 简写
        std::cout << this->total_val << std::endl; // 完整写法
    }

    // 返回对象本身的引用
    const Stock& topval(const Stock& s) const {
        if (s.total_val > total_val)
            return s;    // 返回参数对象
        else
            return *this; // 返回调用对象本身
    }
};
```

4.2 this指针的使用场景

```

class Person {
private:
    std::string name;
    int age;

public:
    // 场景1：参数名与成员变量名冲突
    void setName(std::string name) {
        this->name = name; // 明确指定是成员变量
    }

    // 场景2：返回对象本身，支持链式调用
    Person& setAge(int age) {
        this->age = age;
        return *this;
    }

    Person& setName2(const std::string& name) {
        this->name = name;
        return *this;
    }
};

// 链式调用示例：
Person p;
p.setAge(25).setName2("张三"); // 可以连续调用

```

第五部分：const成员函数

5.1 const成员函数的必要性

```

cpp

class Stock {
public:
    void show();    // 普通成员函数
    void show() const; // const成员函数
};

const Stock readonly_stock("Apple", 100, 150.0);
readonly_stock.show(); // 错误！const对象不能调用非const成员函数

```

5.2 const成员函数的正确使用

```

cpp

```

```
class Stock {
private:
    std::string company;
    long shares;
    double share_val;

public:
    // const成员函数：承诺不修改对象
    void show() const {
        std::cout << "Company: " << company << std::endl;
        std::cout << "Shares: " << shares << std::endl;
        // shares = 100; // 错误! const函数不能修改成员
    }

    // 访问器函数通常是const的
    std::string getCompany() const { return company; }
    long getShares() const { return shares; }
    double getShareVal() const { return share_val; }

    // 修改器函数不能是const的
    void setShares(long s) { shares = s; }
};
```

第六部分：对象数组

6.1 声明和初始化对象数组

cpp

```
// 声明对象数组（需要默认构造函数）
Stock stocks[4];

// 使用构造函数初始化数组
const int STKS = 4;
Stock stocks[STKS] = {
    Stock("NanoSmart", 12, 20.0),
    Stock("Boffo Objects", 200, 2.0),
    Stock("Monolithic Obelisks", 130, 3.25),
    Stock() // 使用默认构造函数
};

// 部分初始化（其余元素用默认构造函数）
Stock partial[10] = {
    Stock("Apple", 100, 150.0),
    Stock("Google", 50, 2000.0)
    // 其余8个元素用默认构造函数初始化
};
```

6.2 使用对象数组

```
cpp

// 遍历对象数组
for (int i = 0; i < STKS; i++) {
    stocks[i].show();
}

// 比较数组中的对象
const Stock* top = &stocks[0]; // 指向第一个元素
for (int i = 1; i < STKS; i++) {
    top = &top->topval(stocks[i]); // 使用指针调用成员函数
}
top->show(); // 显示价值最高的股票
```

第七部分：类作用域

7.1 作用域的概念

```
cpp
```

```

class MyClass {
private:
    int value; // 类作用域

public:
    void setValue(int value) { // 参数value与成员value不同
        this->value = value; // 明确指定成员变量
    }

    void show() const {
        std::cout << value << std::endl; // 类内部可直接使用
    }
};

// 类外部使用
MyClass obj;
obj.setValue(10); // 通过对象访问公有成员
// obj.value = 20; // 错误! 不能直接访问私有成员

```

7.2 类作用域中的常量

```

cpp

class Bakery {
private:
    // 方法1: 使用枚举
    enum { MONTHS = 12 };
    double costs[MONTHS];

    // 方法2: 使用static const (推荐)
    static const int WORKERS = 10;
    std::string workers[WORKERS];

public:
    void showMonths() const {
        std::cout << "Months: " << MONTHS << std::endl;
    }
};

```

7.3 C++11作用域内枚举

```

cpp

```

// 传统枚举的问题

```
enum egg { Small, Medium, Large };
```

```
enum tshirt { Small, Medium, Large }; // 错误! 重复定义
```

// C++11作用域内枚举

```
enum class EggSize { Small, Medium, Large };
```

```
enum class TshirtSize { Small, Medium, Large };
```

// 使用时需要指定作用域

```
EggSize breakfast = EggSize::Large;
```

```
TshirtSize clothing = TshirtSize::Medium;
```

// 不能隐式转换为整数

```
// int x = breakfast; // 错误!
```

```
int x = static_cast<int>(breakfast); // 需要显式转换
```

第八部分：实战示例 - 完整的Stock类

cpp

```
// stock.h - 头文件
#ifndef STOCK_H
#define STOCK_H
#include <string>

class Stock {
private:
    std::string company;
    long shares;
    double share_val;
    double total_val;
    void set_tot() { total_val = shares * share_val; }

public:
    // 构造函数
    Stock();
    Stock(const std::string& co, long n = 0, double pr = 0.0);

    // 析构函数
    ~Stock();

    // 成员函数
    void buy(long num, double price);
    void sell(long num, double price);
    void update(double price);
    void show() const;

    // 比较函数
    const Stock& topval(const Stock& s) const;

    // 访问器函数
    std::string getCompany() const { return company; }
    long getShares() const { return shares; }
    double getShareVal() const { return share_val; }
    double getTotalVal() const { return total_val; }
};

#endif
```

cpp

```
// stock.cpp - 实现文件
```

```
#include "stock.h"
```

```
#include <iostream>
```

```
// 默认构造函数
```

```
Stock::Stock() {  
    company = "no name";  
    shares = 0;  
    share_val = 0.0;  
    total_val = 0.0;  
}
```

```
// 带参数的构造函数
```

```
Stock::Stock(const std::string& co, long n, double pr) {  
    company = co;  
    if (n < 0) {  
        std::cout << "股票数量不能为负，设置为0\n";  
        shares = 0;  
    } else {  
        shares = n;  
    }  
    share_val = pr;  
    set_tot();  
}
```

```
// 析构函数
```

```
Stock::~~Stock() {  
    // 通常为0，除非需要清理动态分配的内存  
}
```

```
// 买入股票
```

```
void Stock::buy(long num, double price) {  
    if (num < 0) {  
        std::cout << "购买数量不能为负\n";  
    } else {  
        shares += num;  
        share_val = price;  
        set_tot();  
    }  
}
```

```
// 卖出股票
```

```
void Stock::sell(long num, double price) {  
    if (num < 0) {  
        std::cout << "卖出数量不能为负\n";  
    } else if (num > shares) {
```

```

        std::cout << "卖出数量不能超过持有数量\n";
    } else {
        shares -= num;
        share_val = price;
        set_tot();
    }
}

// 更新股价
void Stock::update(double price) {
    share_val = price;
    set_tot();
}

// 显示股票信息
void Stock::show() const {
    std::cout << "公司: " << company
        << " 股数: " << shares
        << " 股价: $" << share_val
        << " 总价值: $" << total_val << std::endl;
}

// 比较两个股票对象，返回价值更高的那个
const Stock& Stock::topval(const Stock& s) const {
    if (s.total_val > total_val)
        return s;
    else
        return *this;
}

```

cpp

// main.cpp - 使用示例

```
#include <iostream>
```

```
#include "stock.h"
```

```
int main() {
```

```
    // 创建股票对象
```

```
    Stock stock1("Apple", 100, 150.0);
```

```
    Stock stock2("Google", 50, 2000.0);
```

```
    Stock stock3; // 默认构造函数
```

```
    std::cout << "=== 初始状态 ===\n";
```

```
    stock1.show();
```

```
    stock2.show();
```

```
    stock3.show();
```

```
    // 操作股票
```

```
    std::cout << "\n=== 买入操作 ===\n";
```

```
    stock1.buy(50, 155.0);
```

```
    stock1.show();
```

```
    std::cout << "\n=== 卖出操作 ===\n";
```

```
    stock2.sell(20, 2100.0);
```

```
    stock2.show();
```

```
    // 比较股票价值
```

```
    std::cout << "\n=== 比较股票价值 ===\n";
```

```
    const Stock& top = stock1.topval(stock2);
```

```
    std::cout << "价值更高的股票是: ";
```

```
    top.show();
```

```
    // 对象数组示例
```

```
    std::cout << "\n=== 股票投资组合 ===\n";
```

```
    Stock portfolio[3] = {
```

```
        Stock("Tesla", 75, 800.0),
```

```
        Stock("Microsoft", 200, 300.0),
```

```
        Stock("Amazon", 25, 3000.0)
```

```
    };
```

```
    for (int i = 0; i < 3; i++) {
```

```
        portfolio[i].show();
```

```
    }
```

```
    return 0;
```

```
}
```

第九部分：抽象数据类型(ADT)示例 - 栈

cpp

// stack.h

#ifndef STACK_H

#define STACK_H

typedef unsigned long Item;

class Stack {

private:

enum { MAX = 10 };

Item items[MAX];

int top;

public:

Stack();

bool isempty() **const**;

bool isfull() **const**;

bool push(**const** Item& item);

bool pop(Item& item);

};

#endif

cpp

```
// stack.cpp
#include "stack.h"

Stack::Stack() {
    top = 0;
}

bool Stack::isempty() const {
    return top == 0;
}

bool Stack::isfull() const {
    return top == MAX;
}

bool Stack::push(const Item& item) {
    if (top < MAX) {
        items[top++] = item;
        return true;
    }
    return false;
}

bool Stack::pop(Item& item) {
    if (top > 0) {
        item = items[--top];
        return true;
    }
    return false;
}
```

第十部分：学习要点总结

10.1 核心概念检查清单

- ☐ 理解类和对象的关系
- ☐ 掌握 public 和 private 的区别
- ☐ 会写构造函数和析构函数
- ☐ 理解 this 指针的作用
- ☐ 知道何时使用 const 成员函数
- ☐ 能够创建和使用对象数组
- ☐ 理解类作用域的概念

10.2 常见错误和避免方法

错误类型	错误示例	正确做法
忘记分号	<code>class A {}</code>	<code>class A {};</code>
构造函数有返回类型	<code>void Stock() {}</code>	<code>Stock() {}</code>
const对象调用非const函数	<code>const Stock s; s.show();</code>	<code>void show() const;</code>
默认构造函数带括号	<code>Stock s();</code>	<code>Stock s;</code>
直接访问私有成员	<code>s.shares = 100;</code>	使用公有成员函数

10.3 最佳实践

- 1. **数据隐藏**：将数据成员设为 private
- 2. **接口简洁**：只暴露必要的公有函数
- 3. **const正确性**：不修改对象的函数标记为 const
- 4. **初始化**：优先使用初始化而不是赋值
- 5. **资源管理**：构造函数分配资源，析构函数释放资源

10.4 下一步学习方向

学完本章后，你应该继续学习：

- 运算符重载
- 拷贝构造函数和赋值运算符
- 继承和派生类
- 虚函数和多态
- 模板类

10.5 练习建议

- 1. **基础练习**：实现一个简单的 Rectangle 类
- 2. **进阶练习**：实现一个 BankAccount 类
- 3. **应用练习**：实现一个简单的 Vector 类（类似数组）
- 4. **综合练习**：设计一个学生成绩管理系统

通过这个手册的学习和练习，你将建立起对C++类和对象的完整理解，为后续的C++学习打下坚实基础！